

LangGraph Core Concepts | Agentic AI using LangGraph

1. Introduction

- This video introduces the **core concepts of LangGraph**.
- The focus is on building LLM workflows using **graphs, nodes, edges, and state**.
- It also covers common workflow patterns such as chaining, routing, and parallel execution.

2. What is LangGraph?

- **LangGraph** is a framework for building **stateful, multi-step LLM and agent workflows**.
- It represents a workflow as a **graph**.
- The graph controls how different steps are executed.
- It is especially useful when an application needs:
 - Branching
 - Loops
 - Parallel execution
 - State management
 - Human intervention
 - Complex agent behavior

Simple idea: LangGraph lets you define **how an AI workflow moves from one step to another**.

3. LLM Workflows

An **LLM workflow** is a sequence of operations where an LLM performs one or more tasks.

Example:

User Input → LLM → Process Result → LLM → Final Output
--

Workflows can have different structures, including:

- Sequential execution
- Conditional routing
- Parallel execution
- Iterative execution

LangGraph provides a structured way to represent these workflows.

4. Prompt Chaining

Prompt chaining means using the output of one LLM call as the input for another.

Example:

Prompt 1 → LLM → Output 1 → Prompt 2 → LLM → Final Output

Example use case:

1. Generate an article idea.
2. Use the idea to create an outline.
3. Use the outline to write the article.

Each step depends on the result of the previous step.

5. Routing

Routing means choosing which path the workflow should follow based on the input or result.

Example:

User Query → Classify Query

- Technical → Technical Agent
- Sales → Sales Agent
- Support → Support Agent

The workflow therefore does not always follow the same path.

6. Parallelization

Parallelization means executing multiple independent tasks at the same time.

Example:

User Query

- Search Web
- Search Database
- Search Documents

Once all tasks finish:

Results → Combine → Final Answer

This can make workflows more efficient when tasks do not depend on each other.

7. Orchestrator Workers

This pattern separates **task planning** from **task execution**.

- **Orchestrator:** Decides what tasks need to be performed.
- **Workers:** Execute those individual tasks.

Example:

Orchestrator

- Worker 1: Research topic A
- Worker 2: Research topic B
- Worker 3: Research topic C
- Combine results

This is useful when the number or type of tasks can change dynamically.

8. Evaluator Optimizer

This pattern uses one component to **generate a result** and another to **evaluate and improve it**.

Basic flow:

Generate → Evaluate → Improve → Evaluate → ...

Example:

- LLM generates an answer.
- Evaluator checks its quality.
- If the answer is poor, it is sent back for improvement.
- The process continues until the result meets the required criteria.

This creates an **iterative improvement loop**.

9. Graphs, Nodes and Edges

LangGraph represents workflows as **graphs**.

- **Graph:** The complete workflow.
- **Node:** An individual task or operation.
- **Edge:** Defines the connection and movement between nodes.

Example:

Node A → Node B → Node C

Here:

- A, B, and C are nodes.
- The arrows are edges.
- The complete structure is the graph.

Edges can also represent **conditional paths**.

10. State

State contains the information shared throughout the workflow.

For example:

```
state = {  
  user_query,  
  search_results,  
  draft,  
  final_answer  
}
```

A node can:

1. Read information from the state.
2. Perform its task.
3. Update the state.

The updated state is then available to other nodes.

Simple idea: State is the workflow's **shared memory of its current information**.

11. Reducers

A **reducer** determines how updates from nodes are combined into the existing state.

This becomes particularly important when **multiple nodes update the same state field**, such as during parallel execution.

Example:

Worker 1 → Result A

Worker 2 → Result B

Worker 3 → Result C

A reducer can combine them into:

Results = [A, B, C]

Without an appropriate reducer, updates may overwrite each other instead of being combined.

12. LangGraph Execution Model

The **execution model** describes how LangGraph runs the workflow.

The basic process is:

Start → Execute Node → Update State → Follow Edge → Execute Next Node
--

Depending on the graph, execution can include:

- Sequential nodes
- Conditional branches
- Parallel nodes
- Loops
- State updates
- Multiple execution paths

Key takeaway: LangGraph turns an LLM workflow into a **structured graph where nodes perform tasks, edges control execution, and state carries information between steps.**
